

Beginner-C Bootcamp 7주차 과제

20215789 산업보안학과 김문정

Challenge 1. malloc / free 기초 실습

```
user@user-VirtualBox: ~  
user@user-VirtualBox:~$ vim malloc.c  
user@user-VirtualBox:~$ gcc -g malloc.c -o malloc  
user@user-VirtualBox:~$ ./malloc  
num 값 : 100  
arr 값들 : 10 20 30 40 50  
num 주소 : 0x636b095992a0  
arr 주소 : 0x636b095992c0  
global_var 주소 : 0x636ad53bc010  
local_var 주소 : 0x7ffdf47883ac  
메모리 해제 완료  
user@user-VirtualBox:~$
```

실제로 heap영역에 저장되어있는지 확인하기 위해 Data영역의 전역변수, Stack영역의 지역변수를 선언하여 주소를 출력해보았다.

높은 주소

0x7ffdf47883ac ← local_var (Stack)

빈 공간

0x636b095992c0 ← arr (Heap)

0x636b095992a0 ← num (Heap)

0x636ad53bc010 ← global_var (Data)

Code/Text

낮은 주소

여기서 보통 heap은 위로 증가하여, 낮은 주소 -> 높은 주소 방향으로 증가한다. 그래서 num -> arr순으로 주소가 커진 것을 볼 수 있다.

Stack은 반대로 높은 주소 -> 낮은 주소 방향으로 성장하여 함수 호출이 많아질수록 주소가 감소한다.

또한 heap영역의 num, arr을 보면 주소가 연속적으로 할당된 것을 볼 수 있는데 malloc이 두 변수에 대하여 연속된 값을 할당시켜주는 것은 아니고 우연히 연속적으로 비어있는 공간이 있었기 때문에 num, arr의 주소가 연속적이라는 것을 알 수 있다.

malloc()은 메모리 할당에 실패하면 NULL을 반환한다.

즉, `int *ptr = malloc(sizeof(int));` 에서 메모리가 부족하면 `ptr == NULL` 상태가 되는 것이다.

따라서 `ptr == NULL` 상태인지를 확인하고 NULL이 되면 리턴값을 반환해주어야 한다.

```
if (ptr == NULL) {  
    printf("메모리 할당 실패!\n");  
    return 1;  
}
```

만일 `ptr == NULL` 검사를 하지 않는다면 잘못된 메모리 접근이 가능하다. NULL은 아무 주소도 가리키지 않고 내부적으로 0x0 주소를 의미한다.

즉, `int *ptr = NULL;` 은 ptr이 유효한 메모리를 가리키지 않는다는 것을 의미한다.

그런데 만약 malloc() 실패 시에도 프로그램이 계속 진행되고, `*ptr = 10;` 와 같이 접근하라는 명령이 떨어지면 `*(0x0) = 10;` 의 의미가 되어 메모리 주소 0번지에 접근하라는 명령이 된다.

그러나 운영체제는 0x0 메모리 접근 금지를 걸어두기 때문에, 접근하면 Segmentation Fault, 허용되지 않은 메모리에 접근했다는 오류가 발생한다.

```

user@user-VirtualBox: ~
#include <stdio.h>

int main() {

    int *ptr = NULL;

    *ptr = 10;

    return 0;
}

user@user-VirtualBox:~$ ./segfault
Segmentation fault (core dumped)

```

실제로 저렇게 코드를 짜보면, segfault error가 나오는 것을 알 수 있다.

malloc()으로 할당한 Heap 메모리는 직접 free() 해야만 반환된다.

만약 malloc(...)만 하고 free()를 해주지 않으면 Memory Leak (메모리 누수)가 발생한다.

여기서 Memory Leak이란 프로그램이 메모리를 계속 할당하지만 더 이상 사용하지 않으면서 반환도 안 하는 상태로 쓸모없는 메모리가 계속 점유되고 있는 상황이다.

```

#include <stdio.h>
#include <stdlib.h>

int main() {

    while (1) {

        // Heap 할당
        int *ptr = (int *)malloc(1000);

        // malloc 실패
        if (ptr == NULL) {
            printf("메모리 할당 실패!\n");
            break;
        }

        printf("할당된 주소: %p\n", (void *)ptr);

        // free(ptr); ← Memory Leak

    }

    return 0;
}

```

```
user@user-VirtualBox: ~  
할당된 주소 : 0x5eb5fe4f52e0  
할당된 주소 : 0x5eb5fe4f56d0  
할당된 주소 : 0x5eb5fe4f5ac0  
할당된 주소 : 0x5eb5fe4f5eb0  
할당된 주소 : 0x5eb5fe4f62a0  
할당된 주소 : 0x5eb5fe4f6690  
할당된 주소 : 0x5eb5fe4f6a80  
할당된 주소 : 0x5eb5fe4f6e70  
할당된 주소 : 0x5eb5fe4f7260  
할당된 주소 : 0x5eb5fe4f7650  
할당된 주소 : 0x5eb5fe4f7a40  
할당된 주소 : 0x5eb5fe4f7e30  
할당된 주소 : 0x5eb5fe4f8220  
할당된 주소 : 0x5eb5fe4f8610  
할당된 주소 : 0x5eb5fe4f8a00  
할당된 주소 : 0x5eb5fe4f8df0  
할당된 주소 : 0x5eb5fe4f91e0  
할당된 주소 : 0x5eb5fe4f95d0  
할당된 주소 : 0x5eb5fe4f99c0  
할당된 주소 : 0x5eb5fe4f9db0  
할당된 주소 : 0x5eb5fe4fa1a0  
할당된 주소 : 0x5eb5fe4fa590  
할당된 주소 : 0x5eb5fe4fa980
```

위 코드를 실행하면 사진과 같이 할당된 주소가 계속 출력되는 것을 알 수 있다.

(Memory leak) 그러나 한계점에 도달하면 메모리가 부족해지거나 운영체제가 프로그램을 종료시키거나 malloc이 NULL 반환을 한다는 사실을 알 수 있다.

서버 프로그램에서 누수가 계속되면 Out Of Memory (OOM)이 발생 가능하며 이에 올바른 사용 흐름은

1. malloc()으로 할당
 2. 포인터가 NULL일 때 정의
 3. 사용
 4. free()로 해제(반환)
- 이 된다.

Challenge2. Dangling Pointer 실험

```
user@user-Virt
#include <stdio.h>
#include <stdlib.h>

int main() {

    // Heap
    int *ptr = (int *)malloc(sizeof(int));

    // malloc 실패
    if (ptr == NULL) {
        printf("메모리 할당 실패\n");
        return 1;
    }

    *ptr = 100;

    printf("free 전 값: %d\n", *ptr);
    printf("주소: %p\n", (void *)ptr);

    // 메모리 해제
    free(ptr);

    printf("\n=== free 이후 ===\n");

    // Dangling Pointer
    printf("free 후 값 읽기: %d\n", *ptr);

    // Use After Free
    *ptr = 999;

    printf("free 후 값 쓰기: %d\n", *ptr);

    return 0;
}
```

```
user@user-VirtualBox:~$ vim dangling.c
user@user-VirtualBox:~$ gcc -g dangling.c -o dangling
user@user-VirtualBox:~$ ./dangling
free 전 값: 100
주소: 0x5e76e89922a0

=== free 이후 ===
free 후 값 읽기: -412186222
free 후 값 쓰기: 999
user@user-VirtualBox:~$
```

위 사진과 같이 메모리를 할당하고, 다시 free로 반환했다. 그런데 free로 메모리를 반환한 이후 *ptr로 포인터가 가리키는 값을 읽으려 시도하니 -412186222라는 쓰레기값이 출력되었다. 또 이후 *ptr=999로 값을 넣고 다시 *ptr로 포인터가 가리키는 값을 읽어오니 이번에는 사용자가 넣은 값인 999가 잘 출력되는 것을 확인할 수 있다. 여기서 쓰레기값이 출력된 이유는 이미 os에 반환된 메모리를 읽었기 때문인데 os에 반환되었기 때문에 그 주소의 메모리가 allocator내부데이터, 쓰레기값, 다른 정보 등으로 바뀌어 있었기 때문이다.

하지만 중요한 점은 free이후 값을 쓰고 그 값을 읽는 것이 가능했다는 점인데 이것이 의미하는 것은 해제된 메모리인데도 접근이 가능했다는 것이다.

즉, free 했다고 즉시 메모리가 삭제되는 것은 아니며 allocator가 아직 그 영역을 사용하지 않았기 때문에 우연히 쓰기가 가능했던 것이다.

free로 반환한 메모리는 시스템 입장에서는 빈 공간이고, 나중에 다른 malloc이나 변수가 이 주소를 재사용할 수 있다. 그런데 `*ptr = 999;` 로 덮어쓴 것은, 남의 메모리를 몰래 수정한 것과 비슷한 상황이라고 이해할 수 있다.

이것이 Use After Free (UAF) 취약점인데

공격자는

1. 객체 free 유도
2. 같은 주소 재사용 유도
3. Dangling Pointer로 값 덮어쓰기

를 통해

- 함수 포인터 조작
- 객체 데이터 변조
- 코드 실행

을 하도록 한다.

위 결과를 요약하자면

free 이후 메모리가 해제되었지만 포인터는 여전히 옛 주소를 가리키고 있었고, (주소는 기억하지만 안에 있는 메모리는 해제한 상황) 그 메모리에 접근하자 쓰레기값이 읽혔으며, 쓰기까지 가능했던 상태이다.

gdb로 직접 확인해보면

```
Breakpoint 1, main () at dangling.c:21
21      free(ptr);
(gdb) p ptr
$1 = (int *) 0x5555555592a0
(gdb) x/wx ptr
0x5555555592a0: 0x00000064
(gdb)
```

메모리가 free되기 직전에 bf를 걸고 ptr 주소와 안에 있는 값을 확인해보면 100인 것을 확인할 수 있다.

```
(gdb) next
23      printf("\n=== free 이후 ===\n");
(gdb) x/wx ptr
0x5555555592a0: 0x55555559
```

next로 free를 한 이후 다시 ptr주소와 안에 있는 값을 확인해보면, 주소는 그대로이지만 allocator가 값을 변경해주었다는 것을 알 수 있다.

```
=== free 이후 ===
26      printf("free 후 값 읽기: %d\n", *ptr);
(gdb) next
free 후 값 읽기: 1431655769
29      *ptr = 999;
(gdb) next
31      printf("free 후 값 쓰기: %d\n", *ptr);
(gdb) x/wx ptr
0x5555555592a0: 0x000003e7
(gdb)
```

또한 *ptr=999를 넣어주고 다시 x/wx ptr를 출력해주면 주소값은 그대로지만 0x03e7이라는 10진수 999가 있는 것을 확인할 수 있다.